



Objetivo General

Diseñar e implementar soluciones empresariales aplicando múltiples patrones de diseño en un mismo problema, justificando las decisiones de los patrones.

Competencias a Desarrollar

- Escribir código mantenible y extensible.
- Integran múltiples patrones en un mismo problema.
- Justifican el patrón de diseño.
- Diseñan antes de programar.
- Piensan como arquitectos de software.
- Diseñar soluciones desacopladas y extensibles.
- Modelar sistemas mediante UML.
- Evaluar impacto de diseño en mantenimiento y escalabilidad.
- Integrar patrones estructurales, creacionales y de comportamiento.

Nota: Todos los ejercicios deben tener el diagrama de clases.

Ejercicio 1: Plataforma de Comercio Electrónico Escalable.

El sistema debe soportar **múltiples métodos de pago** que puedan incorporarse dinámicamente sin modificar la lógica central del proceso de compra, lo que hace necesario implementar el patrón **Strategy** para encapsular algoritmos de pago intercambiables (tarjeta, PayPal, criptomonedas) junto con el patrón **Factory Method** que centralice la creación de estos objetos, permitiendo extender el sistema a nuevos métodos como transferencias bancarias o pagos móviles sin violar el principio Open/Closed. Adicionalmente, la plataforma requiere un **sistema flexible de descuentos** que puedan combinarse de forma acumulativa (porcentajes, montos fijos, cupones promocionales), por lo que el patrón **Decorator** resulta ideal para envolver objetos de pedido base con diferentes tipos de descuentos sin alterar su estructura original. En cuanto a la **logística de envíos**, se necesita calcular costos según diferentes estrategias (estándar, exprés, internacional), nuevamente aplicando el patrón **Strategy** para permitir cambios en tiempo de ejecución según preferencias del usuario o condiciones del pedido. Finalmente, el sistema debe **notificar eventos** del ciclo de compra (confirmación, envío, entrega) a través de múltiples canales (email, SMS, push notifications), lo que justifica el uso del patrón **Observer** para desacoplar el procesamiento de pedidos de los mecanismos de notificación, permitiendo agregar nuevos canales sin modificar la lógica principal. Esta combinación de patrones asegura una arquitectura escalable, mantenible y preparada para futuros requerimientos como nuevos métodos de pago, promociones especiales o integración con servicios externos de mensajería y logística.

Objetivo: Diseñar un sistema de ventas que permita incorporar dinámicamente métodos de pago, descuentos, envíos y notificaciones sin modificar el núcleo del



sistema.

Competencias específicas

- Aplicar Strategy y Decorator correctamente.
- Diseñar creación desacoplada con Factory.
- Implementar comunicación desacoplada con Observer.

Contexto Una empresa de retail necesita:

- Múltiples métodos de pago.
- Diferentes tipos de envío.
- Cálculo dinámico de descuentos.
- Notificaciones post-compra.

Requerimientos técnicos el sistema debe permitir:

- Agregar nuevos métodos de pago sin modificar código existente.
- Aplicar promociones dinámicamente.
- Notificar por diferentes canales.

Patrones involucrados

- **Strategy** (métodos de pago, descuentos)
- **Factory Method** (creación de pagos/envíos)
- **Observer** (notificaciones post-compra)
- **Decorator** (descuentos acumulativos)

Preguntas de reflexión

- ¿Qué ocurre si el sistema crece a 15 métodos de pago?
- ¿Cómo evitar modificar la clase principal al agregar promociones?
- ¿Qué ventajas ofrece Observer frente a llamadas directas?
- ¿Qué patrón garantiza apertura para extensión?
- ¿Qué riesgos hay si se mezclan responsabilidades?

Ejercicio 2: Sistema de Monitoreo de Microservicios.

El sistema debe garantizar una configuración centralizada y consistente para todos los componentes de monitoreo, donde los umbrales de alerta (como el 80% de uso de CPU) sean **accesibles globalmente sin duplicación de instancias**, lo que hace indispensable el **patrón Singleton** para la clase MonitoringConfig, asegurando que todos los microservicios consulten y modifiquen los mismos parámetros de configuración en tiempo real. Ante la necesidad de **notificaciones proactivas y multicanal**, el sistema requiere que los microservicios notifiquen automáticamente a diferentes sistemas de alerta cuando se superen los umbrales establecidos, por lo que el **patrón Observer** permite desacoplar los monitores de los mecanismos de



notificación (EmailAlert, SlackAlert), facilitando la incorporación futura de nuevos canales como **SMS o Microsoft Teams sin modificar la lógica principal de monitoreo**. Dado que el sistema debe **integrarse con APIs externas de monitoreo** que utilizan formatos de datos heterogéneos (por ejemplo, una API legada que devuelve {"cpu_usage_percent": 85}), el **patrón Adapter** resulta fundamental para normalizar estas interfaces dispares, permitiendo que el núcleo del sistema consuma métricas de manera uniforme a través del método get_cpu_usage(). Finalmente, para **ofrecer una interfaz simplificada que abstraiga la complejidad del subsistema de monitoreo**, el **patrón Facade** centraliza el registro de servicios y la ejecución del monitoreo periódico, proporcionando un punto único de entrada que coordina la obtención de métricas, la verificación de umbrales y la notificación de alertas, ocultando la complejidad subyacente a los equipos de operaciones que consumirán esta funcionalidad. **Esta arquitectura basada en patrones** garantiza un sistema de monitoreo extensible, mantenible y preparado para escalar conforme crezca el ecosistema de microservicios.

Objetivo: Diseñar una plataforma que permita agregar nuevas métricas y tipos de alerta sin afectar el sistema base.

Competencias específicas

- Implementar Observer para alertas.
- Centralizar configuración con Singleton.
- Integrar APIs externas usando Adapter.
- Simplificar subsistemas con Facade.

Contexto Una empresa fintech necesita monitorear:

- Estado de servicios.
- Logs.
- Alertas en tiempo real.

Requerimientos Técnicos: El sistema debe permitir

- Agregar nuevos tipos de métricas.
- Enviar alertas por diferentes medios.
- Centralizar la configuración.

Patrones involucrados

- **Observer** (alertas)
- **Singleton** (configuración central)
- **Adapter** (integración con APIs externas)
- **Facade** (interfaz simplificada de monitoreo)

Preguntas de reflexión

- ¿Qué problema resuelve Singleton aquí?



- ¿Cómo desacoplar servicios externos?
- ¿Por qué Facade mejora mantenibilidad?
- ¿Cómo escalar el sistema a 100 microservicios?
- ¿Dónde se produciría alto acoplamiento si no se aplican patrones?

Ejercicio 3: Plataforma de LMS Universitaria.

El sistema debe gestionar **múltiples tipos de usuarios** (estudiantes, profesores, administradores) con roles y permisos diferenciados, donde la creación de estos objetos debe centralizarse para garantizar consistencia y facilitar la incorporación de nuevos roles en el futuro, por lo que el patrón **Factory Method** implementado en UserFactory permite encapsular la lógica de instanciación según el tipo de usuario solicitado, manteniendo el código cliente desacoplado de las clases concretas. Para asegurar una **configuración unificada** de la plataforma (nombre de la institución, políticas académicas, parámetros globales), el patrón **Singleton** en LMSConfig garantiza que todos los módulos del sistema accedan a la misma instancia de configuración, evitando inconsistencias en datos críticos como el nombre de la universidad o los períodos académicos vigentes. Ante la necesidad de **comunicación asincrónica y notificaciones** entre los diferentes actores del sistema, el patrón **Observer** permite que los cursos notifiquen automáticamente a estudiantes y profesores sobre anuncios, cambios de calendario o eventos académicos a través de múltiples canales (email, notificaciones push en app móvil), desacoplando la lógica del curso de los mecanismos de notificación. Considerando la **diversidad de métodos de evaluación** (promedio simple, ponderado por porcentajes, evaluación basada en rúbricas), el patrón **Strategy** encapsula diferentes algoritmos de cálculo de calificaciones en clases separadas, permitiendo que los profesores seleccionen dinámicamente el método más adecuado para cada curso o actividad sin modificar la lógica principal del sistema. Para la **persistencia y recuperación de datos**, el patrón **Repository** abstrae el acceso a la fuente de datos (actualmente en memoria, pero preparado para migrar a bases de datos relacionales o NoSQL), proporcionando una interfaz consistente para operaciones CRUD sobre usuarios, cursos y calificaciones, lo que facilita pruebas unitarias y cambios en la capa de persistencia sin afectar el dominio. Finalmente, el patrón **Facade** en LMSFacade ofrece una interfaz simplificada que unifica todas estas funcionalidades (registro de usuarios, creación de cursos, gestión de evaluaciones) para los clientes del sistema, ocultando la complejidad subyacente de las interacciones entre fábricas, repositorios, estrategias y observadores, y proporcionando un punto de entrada único que los equipos de desarrollo de frontend o servicios REST pueden consumir de manera intuitiva. Esta arquitectura basada en patrones garantiza una plataforma LMS extensible, mantenible y preparada para escalar conforme crezca la comunidad universitaria y se agreguen nuevas funcionalidades académicas.



Objetivo: Diseñar una plataforma académica modular que permita plugins de evaluación y múltiples formatos de reporte.

Competencias específicas

- Diseñar extensibilidad mediante Factory.
- Implementar Strategy para generación de reportes.
- Controlar acceso con Proxy.
- Gestionar eventos con Observer.

Contexto La universidad necesita:

- Gestión de cursos.
- Entregas.
- Evaluaciones.
- Reportes académicos.

Requerimientos técnicos el sistema debe permitir

- Generar reportes en múltiples formatos.
- Permitir plugins de evaluación.
- Controlar permisos por rol.

Patrones involucrados

- **Factory (evaluaciones)**
- **Strategy (formatos de reporte)**
- **Proxy (control de acceso)**
- **Observer (notificación de calificaciones)**

Preguntas de reflexión

- ¿Cómo se agregaría un nuevo tipo de evaluación?
- ¿Qué ventajas tiene desacoplar formato de reporte?
- ¿Por qué Proxy es mejor que condicionales de rol?
- ¿Cómo evitar que el sistema central crezca excesivamente?
- ¿Cómo impacta el diseño en mantenibilidad a largo plazo?

Ejercicio 4: Sistema de Gestión Hospitalaria.

El sistema debe gestionar la **diversidad de actores** que interactúan en el entorno hospitalario (pacientes, médicos, personal administrativo), cada uno con roles y permisos específicos, por lo que el patrón **Factory Method** implementado



en PersonFactory permite centralizar la creación de estos objetos, garantizando que la lógica de instanciación sea consistente y extensible para futuros roles como enfermeros o especialistas externos sin modificar el código cliente. Dado que el hospital requiere una **configuración unificada** (nombre de la institución, horarios de atención, políticas internas) accesible desde todos los módulos del sistema, el patrón **Singleton** en HospitalConfig asegura que exista una única instancia global de configuración, evitando duplicaciones y garantizando que cualquier cambio en los parámetros generales se refleje inmediatamente en todo el sistema. Ante la necesidad crítica de **notificaciones oportunas** para confirmación de citas, recordatorios de medicamentos o alertas de emergencia, el patrón **Observer** permite que eventos como la confirmación de una cita notifiquen automáticamente a los pacientes y médicos a través de múltiples canales (email, SMS), desacoplando la lógica de las citas de los mecanismos de notificación y facilitando la incorporación futura de nuevos canales como notificaciones push en una app móvil. Considerando la **complejidad de los procesos de facturación**, que varían según el tipo de paciente (particular, con seguro, convenios institucionales), el patrón **Strategy** encapsula los diferentes algoritmos de cálculo de costos en clases separadas (InsuranceBilling, PrivateBilling, AgreementBilling), permitiendo que el sistema seleccione dinámicamente la estrategia adecuada según el perfil del paciente sin modificar la lógica central de facturación. Para la **integración con sistemas externos de aseguradoras**, que pueden tener interfaces y formatos de datos heterogéneos, el patrón **Adapter** normaliza la comunicación al envolver APIs externas no modificables (como ExternalInsuranceAPI) y proporcionar una interfaz uniforme (InsuranceAdapter) que el sistema hospitalario puede consumir sin acoplarse a implementaciones específicas de terceros. En cuanto a la **persistencia de datos clínicos y administrativos**, el patrón **Repository** abstrae el acceso a la fuente de datos (actualmente en memoria, pero preparado para migrar a bases de datos relacionales o historias clínicas electrónicas), proporcionando una interfaz consistente para operaciones CRUD sobre pacientes y otros registros, lo que facilita pruebas unitarias y cambios en la capa de persistencia sin afectar el dominio. Finalmente, el patrón **Facade** en HospitalFacade ofrece una interfaz simplificada que unifica todas estas funcionalidades (registro de pacientes, agendamiento de citas, facturación) para los clientes del sistema, ocultando la complejidad subyacente de las interacciones entre fábricas, repositorios, adaptadores, estrategias y observadores, y proporcionando un punto de entrada único que los equipos de desarrollo de interfaces administrativas, portales de pacientes o aplicaciones móviles pueden consumir de manera intuitiva. Esta arquitectura basada en patrones garantiza un sistema hospitalario robusto, escalable y preparado para cumplir con los exigentes requisitos de disponibilidad, confidencialidad y eficiencia propios del sector salud.

Objetivo: Diseñar un sistema modular que gestione historia clínica, notificaciones e integración externa.

Competencias específicas



- Separar persistencia con Repository.
- Desacoplar eventos clínicos.
- Integrar sistemas legacy.
- Simplificar acceso a módulos.

Contexto Hospital que maneja:

- Pacientes
- Historial clínico
- Facturación
- Notificaciones médicas

Requerimientos técnicos:

- Integración con sistemas externos.
- Registro automático de cambios clínicos.
- Separación entre lógica médica y presentación.

Patrones involucrados

- **Observer** (actualizaciones clínicas)
- **Adapter** (sistemas externos)
- **Facade** (módulo hospitalario)
- **Repository** (acceso a datos)

Preguntas de reflexión

- ¿Por qué separar acceso a datos?
- ¿Qué problema resuelve Adapter?
- ¿Cómo evitar acoplamiento entre módulos?
- ¿Qué patrón mejora claridad arquitectónica?
- ¿Cómo facilitar auditorías y mantenimiento?

Ejercicio 5: Motor de Procesamiento de Pagos Internacionales.

El sistema debe garantizar una **configuración unificada y consistente** para parámetros globales como la moneda predeterminada (USD), tasas de cambio base y umbrales de seguridad, por lo que el patrón **Singleton** implementado en PaymentConfig asegura que todos los módulos del motor de pagos accedan a la misma instancia de configuración, evitando inconsistencias críticas en operaciones financieras transfronterizas. Considerando la **diversidad de métodos de pago** que deben soportarse en diferentes regiones (tarjetas de crédito, transferencias bancarias SEPA/ACH, criptomonedas, billeteras digitales), el patrón **Factory Method** en PaymentFactory centraliza la creación de estos objetos, permitiendo incorporar nuevos métodos de pago específicos por país sin modificar la lógica central



del procesador. Para el **cálculo dinámico de comisiones**, que varían según el tipo de cliente (estándar, premium) y la naturaleza internacional de la transacción, el patrón **Strategy** encapsula los diferentes algoritmos de comisión (StandardCommission, PremiumCommission, InternationalCommission), facilitando la selección en tiempo de ejecución basada en el perfil del comercio y las regulaciones locales. Ante la necesidad crítica de **validación antifraude multicapa**, el patrón **Chain of Responsibility** permite encadenar verificaciones progresivas (montos sospechosos, países de alto riesgo, patrones de comportamiento anómalos), donde cada eslabón puede procesar la solicitud o pasarla al siguiente, proporcionando flexibilidad para agregar nuevas reglas de seguridad sin acoplar la lógica de validación. Para la **integración con múltiples pasarelas de pago externas** (Stripe, PayPal, MercadoPago) que presentan interfaces heterogéneas, el patrón **Adapter** normaliza la comunicación, envolviendo APIs externas no modificables y proporcionando una interfaz uniforme que el motor principal puede consumir independientemente del proveedor subyacente. En cuanto a la **notificación de eventos transaccionales**, el patrón **Observer** desacopla el procesamiento de pagos de los mecanismos de notificación, permitiendo que cada transacción exitosa o fallida notifique automáticamente a comerciantes y clientes a través de múltiples canales (email, SMS, webhooks) sin modificar la lógica central. Finalmente, el patrón **Facade** en PaymentFacade ofrece una interfaz simplificada que unifica todas estas capacidades (selección de método de pago, cálculo de comisiones, validación antifraude, procesamiento con gateway externo y notificaciones), ocultando la complejidad subyacente y proporcionando un punto de entrada único que los equipos de integración pueden consumir mediante una simple llamada pay(method_type, amount, country). Esta arquitectura basada en patrones garantiza un motor de pagos internacional robusto, extensible y preparado para escalar conforme se expanda a nuevos mercados, con nuevos métodos de pago, regulaciones locales y proveedores de servicios financieros.

Objetivo: Diseñar un motor flexible que valide, procese y adapte pagos según país y regulación.

Competencias específicas

- Encadenar validaciones con Chain of Responsibility.
- Definir flujo base con Template Method.
- Crear procesadores por región con Factory.
- Aplicar Strategy para conversión monetaria.

Contexto Empresa global procesa pagos con:

- Diferentes monedas.
- Diferentes regulaciones.
- Validaciones antifraude.

Requerimientos técnicos

- Aplicar validaciones encadenadas.



- Convertir monedas dinámicamente.
- Crear procesadores según país.

Patrones involucrados

- **Chain of Responsibility** (validaciones)
- **Factory Method** (procesadores por país)
- **Strategy** (conversión de moneda)
- **Template Method** (flujo general del pago)

Preguntas de reflexión

- ¿Qué ventaja tiene encadenar validaciones?
- ¿Cómo evitar múltiples if por país?
- ¿Qué parte del algoritmo debe ser invariable?
- ¿Cómo escalar a nuevos países?
- ¿Qué patrón reduce duplicación de lógica?

Ejercicio 6: Sistema de Reservas para Aerolínea.

El sistema debe implementar una **estructura flexible para la construcción de reservas** que permita ensamblar objetos complejos paso a paso, considerando que una reserva puede incluir pasajero, vuelo, precio base, servicios adicionales y preferencias específicas; por ello, el patrón **Builder** en ReservationBuilder proporciona un mecanismo fluido y controlado para construir objetos Reservation con diferentes configuraciones, evitando constructores telescópicos y garantizando que el objeto resultante sea consistente antes de ser utilizado. Ante la necesidad de **calcular tarifas dinámicamente** según el tipo de clase (económica, premium, primera clase), temporada del año o anticipación de la compra, el patrón **Strategy** encapsula los diferentes algoritmos de cálculo de precios (EconomyPricing, PremiumPricing), permitiendo que cada reserva seleccione la estrategia adecuada en tiempo de ejecución sin modificar la lógica central de la reserva. Considerando que las reservas atraviesan **diferentes estados** durante su ciclo de vida (pendiente, confirmada, cancelada, check-in realizado, abordada), el patrón **State** modela estos comportamientos específicos mediante clases separadas (PendingState, ConfirmedState, CancelledState), permitiendo que la reserva cambie su comportamiento cuando su estado interno cambia, lo que facilita la implementación de reglas de negocio como "solo se pueden modificar reservas en estado pendiente" o "las reservas canceladas no generan cargos adicionales". Para mantener informados a los **pasajeros y agentes de viaje** sobre cambios en el estado de las reservas (confirmación, cancelación, cambios de horario), el patrón **Observer** desacopla la lógica de la reserva de los mecanismos de notificación, permitiendo que múltiples observadores



(EmailNotifier, SMSNotifier, AppNotifier) sean notificados automáticamente cuando ocurren eventos relevantes, sin acoplar el núcleo del sistema a implementaciones específicas de mensajería. Finalmente, la **interfaz unificada** proporcionada por el builder, combinada con la flexibilidad de las estrategias de precios, la máquina de estados y el sistema de notificaciones, crea una arquitectura cohesiva donde los objetos reserva se construyen de manera controlada, responden dinámicamente a cambios de contexto y mantienen informados a todos los interesados, cumpliendo con los exigentes requisitos de disponibilidad, confiabilidad y escalabilidad propios de los sistemas de reservas aerocomerciales. Esta arquitectura basada en patrones garantiza un sistema extensible, mantenible y preparado para incorporar nuevos tipos de tarifas, estados adicionales (como overbooking o lista de espera) y canales de notificación emergentes sin afectar la estabilidad del núcleo del sistema.

Objetivo: Diseñar un sistema que gestione reservas complejas con cambios dinámicos de estado y cálculo flexible de precios.

Competencias específicas

- Modelar comportamiento con State.
- Implementar precios dinámicos con Strategy.
- Construir objetos complejos con Builder.
- Notificar eventos con Observer.

Contexto Debe manejar:

- Reservas.
- Asientos.
- Pagos.
- Cancelaciones.

Requerimientos técnicos

- Calcular precios dinámicos.
- Notificar cambios.
- Permitir upgrades.

Patrones involucrados

- **Strategy** (cálculo de precio)
- **Observer** (notificaciones)
- **State** (estado de reserva)
- **Builder** (construcción de reserva compleja)

Preguntas de reflexión

- ¿Qué problemas genera usar múltiples condicionales para estados?
- ¿Cómo facilita Builder la creación de reservas complejas?
- ¿Qué ocurre si cambian las reglas de precio?



- ¿Cómo desacoplar notificaciones?
- ¿Qué patrón mejora cohesión en cambios de estado?

Ejercicio 7: Sistema de Inventario Inteligente.

El sistema requiere una configuración centralizada y consistente para parámetros críticos como el umbral mínimo de stock (actualmente 10 unidades) que dispara alertas y reórdenes automáticas, por lo que el patrón Singleton implementado en InventoryConfig garantiza que todos los módulos del sistema (gestión de productos, monitoreo, reabastecimiento) accedan a la misma instancia de configuración, permitiendo ajustes en tiempo real sin riesgos de inconsistencia. Ante la diversidad de categorías de productos que maneja un inventario moderno (electrónica, alimentos, ropa, etc.), cada una con comportamientos y necesidades específicas de almacenamiento, el patrón Factory Method en ProductFactory centraliza la creación de objetos Product, encapsulando la lógica de instanciación según el tipo y facilitando la incorporación de nuevas categorías (como productos perecederos o congelados) sin modificar el código cliente. Para la detección proactiva de stock bajo y la comunicación inmediata a los responsables, el patrón Observer desacopla la lógica de monitoreo de los mecanismos de alerta (EmailAlert, SMSAlert), permitiendo que el InventoryManager notifique automáticamente a múltiples canales cuando un producto cae por debajo del umbral configurado, con la flexibilidad de agregar nuevos observadores (como notificaciones en Slack o paneles de control en tiempo real) sin alterar el núcleo del sistema. Considerando la complejidad de las decisiones de reabastecimiento, que pueden basarse en criterios fijos, demanda histórica, estacionalidad o tendencias de mercado, el patrón Strategy encapsula los diferentes algoritmos de reorden (FixedReorderStrategy, DemandBasedReorderStrategy, SeasonalReorderStrategy), permitiendo que el sistema seleccione dinámicamente la estrategia más adecuada para cada categoría de producto o incluso cambiar la estrategia en tiempo de ejecución según el contexto (por ejemplo, usar estrategia estacional durante diciembre). Para la integración con sistemas externos de proveedores, que pueden tener interfaces y protocolos heterogéneos, el patrón Adapter normaliza la comunicación envolviendo APIs externas no modificables (ExternalSupplierAPI) y proporcionando una interfaz uniforme (SupplierAdapter) que el motor de inventario puede consumir independientemente del proveedor, facilitando cambios de proveedor o integración con múltiples fuentes sin afectar la lógica central. En cuanto a la persistencia de datos de inventario, el patrón Repository abstrae el acceso a la fuente de datos (actualmente en memoria, pero preparado para migrar a bases de datos relacionales o sistemas ERP empresariales), proporcionando una interfaz consistente para operaciones CRUD sobre productos, lo que facilita pruebas unitarias y cambios en la capa de persistencia sin afectar el dominio. Finalmente, el



patrón Facade en InventoryFacade ofrece una interfaz simplificada que unifica todas estas capacidades (registro de productos, configuración de umbrales, monitoreo automático, generación de alertas y reórdenes a proveedores), ocultando la complejidad subyacente de las interacciones entre fábricas, repositorios, adaptadores, estrategias y observadores, y proporcionando un punto de entrada único que los equipos de operaciones y desarrollo pueden consumir mediante operaciones intuitivas como `register_product()` y `monitor_inventory()`. Esta arquitectura basada en patrones garantiza un sistema de inventario inteligente, escalable y preparado para evolucionar con las necesidades cambiantes del negocio, incorporando nuevas categorías de productos, estrategias de reabastecimiento basadas en machine learning, integración con múltiples proveedores y canales de alerta emergentes sin comprometer la estabilidad del núcleo del sistema.

Objetivo: Desarrollar un sistema que automatice reposición y se integre con múltiples proveedores.

Competencias específicas

- Adaptar APIs externas.
- Diseñar estrategias de reposición.
- Generar alertas desacopladas.
- Simplificar acceso con Facade.

Contexto Empresa logística requiere:

- Control de stock.
- Alertas automáticas.
- Integración con proveedores.

Requerimientos técnicos

- Adaptarse a diferentes proveedores.
- Generar órdenes automáticamente.
- Permitir múltiples estrategias de reposición.

Patrones involucrados

- **Adapter** (proveedores externos)
- **Observer** (alertas de stock)
- **Strategy** (reposiciones)
- **Facade** (interfaz simplificada del inventario)

Preguntas de reflexión

- ¿Cómo integrar nuevos proveedores sin modificar el sistema?
- ¿Por qué separar estrategia de reposición?
- ¿Qué ocurre si no se usa Observer?



- ¿Cómo reducir complejidad en el módulo principal?
- ¿Qué patrón ayuda a ocultar subsistemas?

Ejercicio 8: Plataforma de Gestión de Proyectos.

El sistema requiere una **configuración global unificada** que establezca parámetros transversales como el límite máximo de tareas por proyecto (actualmente 100), políticas de privacidad y configuraciones de integraciones, por lo que el patrón **Singleton** implementado en AppConfig garantiza que todos los módulos accedan a la misma instancia de configuración, permitiendo ajustes centralizados sin riesgos de inconsistencia en un entorno donde múltiples equipos y proyectos coexisten simultáneamente. Ante la **diversidad de tipos de proyectos** que pueden gestionarse (desarrollo de software, campañas de marketing, investigación, construcción), cada uno con flujos de trabajo y artefactos específicos, el patrón **Factory Method** en ProjectFactory centraliza la creación de objetos Project, encapsulando la lógica de instanciación según el tipo y facilitando la incorporación de nuevas categorías de proyectos sin modificar el código cliente que los consume. Considerando la **complejidad en la creación de tareas**, que pueden incluir múltiples campos opcionales (título, descripción, prioridad, responsable, fechas, etiquetas, dependencias), el patrón **Builder** en TaskBuilder proporciona un mecanismo fluido y controlado para construir objetos Task paso a paso, evitando constructores con múltiples parámetros y permitiendo la creación de tareas con diferentes configuraciones según las necesidades del proyecto. Dado que las tareas atraviesan **diferentes estados** durante su ciclo de vida (pendiente, en progreso, en revisión, bloqueada, completada), el patrón **State** modela estos comportamientos específicos mediante clases separadas (ToDoState, InProgressState, DoneState), permitiendo que cada tarea cambie su comportamiento cuando su estado interno cambia y facilitando la implementación de reglas de negocio como "solo las tareas en progreso pueden ser bloqueadas" o "las tareas completadas no pueden modificarse". Para la **organización inteligente del trabajo**, que requiere diferentes criterios de priorización según el contexto (alta prioridad primero, FIFO, por fecha límite, por dependencias), el patrón **Strategy** encapsula los diferentes algoritmos de priorización (HighPriorityFirst, FIFO), permitiendo que el servicio de proyectos seleccione dinámicamente la estrategia adecuada o incluso que cada equipo configure su propia estrategia sin modificar la lógica central. En cuanto a la **comunicación de eventos** a los interesados, el patrón **Observer** desacopla la lógica de negocio de los mecanismos de notificación, permitiendo que eventos como creación de proyectos, asignación de tareas o cambios de estado notifiquen automáticamente a múltiples canales (EmailNotification, SlackNotification) sin acoplar el núcleo del sistema a implementaciones específicas de mensajería. Para la **persistencia de datos**, el patrón **Repository** abstrae el acceso a la fuente de datos (actualmente en memoria, pero preparado para migrar a bases de datos relacionales o documentales),



proporcionando una interfaz consistente para operaciones CRUD sobre proyectos y tareas, lo que facilita pruebas unitarias y cambios en la capa de persistencia sin afectar el dominio. Finalmente, el patrón **Facade** en ProjectManagementFacade ofrece una interfaz simplificada que unifica todas estas capacidades (creación de proyectos con fábrica, construcción de tareas con builder, gestión de estados, priorización estratégica, notificaciones y persistencia), ocultando la complejidad subyacente y proporcionando un punto de entrada único que los equipos de desarrollo de interfaces de usuario, APIs REST o integraciones con herramientas externas pueden consumir mediante operaciones intuitivas. Esta arquitectura basada en patrones garantiza una plataforma de gestión de proyectos robusta, extensible y preparada para escalar conforme crezca la organización, incorporando nuevos tipos de proyectos, flujos de trabajo personalizados, estrategias de priorización basadas en machine learning y canales de notificación emergentes sin comprometer la estabilidad del núcleo del sistema.

Objetivo Diseñar una plataforma con flujo dinámico de estados y generación flexible de reportes.

Competencias específicas

- Implementar State en flujo de tareas.
- Diseñar reportes configurables.
- Integrar herramientas externas.
- Implementar notificaciones desacopladas.

Contexto Empresa tecnológica necesita:

- Gestión de tareas.
- Estados de proyecto.
- Notificaciones.
- Reportes.

Requerimientos técnicos

- Flujo de estados configurable.
- Generación flexible de reportes.
- Integración con Slack o correo.

Patrones involucrados

- **State** (estado del proyecto)
- **Observer** (notificaciones)
- **Strategy** (reportes)
- **Adapter** (integraciones externas)

Preguntas de reflexión

- ¿Por qué usar State en lugar de múltiples condicionales?



- ¿Cómo hacer reportes extensibles?
- ¿Qué ventaja tiene Adapter?
- ¿Cómo afecta el diseño a la escalabilidad?
- ¿Qué patrón facilita mantenimiento futuro?

Ejercicio 9: Plataforma Streaming Empresarial.

El sistema requiere una configuración global unificada que establezca parámetros transversales como la calidad de reproducción predeterminada (actualmente 720p), políticas de ancho de banda corporativo y restricciones geográficas, por lo que el patrón Singleton implementado en StreamingConfig garantiza que todos los módulos de la plataforma accedan a la misma instancia de configuración, permitiendo ajustes centralizados sin riesgos de inconsistencia en un entorno donde múltiples departamentos y sedes consumen contenido simultáneamente. Ante la diversidad de formatos de contenido que debe gestionar una plataforma empresarial (películas corporativas, transmisiones en vivo de eventos, webinars formativos, cursos bajo demanda), cada uno con comportamientos y requisitos específicos de reproducción, el patrón Factory Method en ContentFactory centraliza la creación de objetos Content, encapsulando la lógica de instanciación según el tipo y facilitando la incorporación de nuevos formatos (como podcasts internos o videoconferencias grabadas) sin modificar el código cliente que los consume. Considerando la variabilidad en la calidad de streaming según las condiciones de red, el tipo de suscripción del empleado o la criticidad del contenido, el patrón Strategy encapsula los diferentes algoritmos de calidad (HDStrategy, FullHDStrategy, UltraHDStrategy), permitiendo que el StreamingManager seleccione dinámicamente la estrategia adecuada en tiempo de ejecución, optimizando el uso del ancho de banda corporativo sin afectar la experiencia del usuario. Dado que el reproductor multimedia atraviesa diferentes estados operativos durante la sesión de visualización (reproduciendo, pausado, detenido, en búsqueda), el patrón State modela estos comportamientos específicos mediante clases separadas (PlayingState, PausedState, StoppedState), permitiendo que el reproductor cambie su comportamiento cuando su estado interno cambia y facilitando la implementación de reglas como "no se puede pausar un contenido ya detenido" o "al reanudar, mantener la posición exacta". Para la comunicación de eventos críticos a los administradores y usuarios, el patrón Observer desacopla la lógica de reproducción de los mecanismos de notificación, permitiendo que eventos como inicio de transmisión, fallos de conexión o disponibilidad de nuevo contenido notifiquen automáticamente a múltiples canales (EmailNotifier, SMSNotifier) sin acoplar el núcleo del sistema a implementaciones específicas de mensajería corporativa. Ante la necesidad de funcionalidades premium diferenciadas según el perfil del empleado (sin anuncios, descarga para visualización offline, acceso prioritario a estrenos), el patrón Decorator permite añadir



dinámicamente responsabilidades adicionales al servicio base de streaming (NoAdsDecorator, DownloadDecorator), envolviendo objetos StreamingService de manera transparente y permitiendo combinaciones flexibles sin modificar la estructura original del servicio. Para la integración con proveedores externos de distribución de contenido (CDN como Akamai, CloudFront o redes internas de caching), que presentan interfaces heterogéneas, el patrón Adapter normaliza la comunicación envolviendo APIs externas no modificables (ExternalCDN) y proporcionando una interfaz uniforme (CDNAdapter) que el motor de streaming puede consumir independientemente del proveedor, facilitando cambios de CDN o integración con múltiples proveedores según la región geográfica. Finalmente, el patrón Facade en StreamingFacade ofrece una interfaz simplificada que unifica todas estas capacidades (creación de contenido con fábrica, aplicación de calidad estratégica, gestión de estados del reproductor, notificaciones, funcionalidades premium mediante decoradores y entrega vía CDN adaptada), ocultando la complejidad subyacente y proporcionando un punto de entrada único que los equipos de desarrollo de aplicaciones de escritorio, móviles o interfaces web pueden consumir mediante operaciones intuitivas como `start_stream()`. Esta arquitectura basada en patrones garantiza una plataforma de streaming empresarial robusta, extensible y preparada para escalar conforme crezca la organización, incorporando nuevos formatos de contenido, estrategias de calidad adaptativas basadas en machine learning, funcionalidades premium adicionales y proveedores de CDN emergentes sin comprometer la estabilidad del núcleo del sistema.

Objetivo: Diseñar una plataforma que soporte múltiples formatos, control de acceso y funcionalidades premium.

Competencias específicas

- Diseñar algoritmos intercambiables.
- Implementar control de acceso con Proxy.
- Crear objetos con Factory.
- Agregar funcionalidades dinámicas con Decorator.

Contexto Empresa de capacitación interna que transmite:

- Videos en vivo
- Grabaciones
- Diferentes niveles de calidad

Requerimientos técnicos

- Soportar múltiples formatos.
- Permitir agregar nuevos algoritmos de compresión.
- Controlar acceso según rol.

Patrones involucrados



- Strategy (algoritmos de compresión)
- Proxy (control de acceso)
- Factory (creación de reproductores)
- Decorator (funciones premium)

Preguntas de reflexión

- ¿Cómo agregar nuevos formatos sin romper el sistema?
- ¿Por qué Proxy es mejor que validaciones directas?
- ¿Qué ventaja tiene Decorator frente a herencia?
- ¿Cómo evitar clases gigantes?
- ¿Cómo escalar la arquitectura?

Ejercicio 10: Motor de Recomendación para E-commerce.

El sistema requiere una configuración global unificada que establezca parámetros transversales como la estrategia de recomendación predeterminada (actualmente "collaborative"), umbrales de similitud y ventanas de tiempo para análisis de comportamiento, por lo que el patrón Singleton implementado en RecommendationConfig garantiza que todos los módulos del motor accedan a la misma instancia de configuración, permitiendo ajustes centralizados sin riesgos de inconsistencia en un entorno donde múltiples usuarios y sesiones generan recomendaciones simultáneamente. Ante la diversidad de algoritmos de recomendación que pueden implementarse (filtrado colaborativo, basado en contenido, híbrido, basado en tendencias, etc.), cada uno con diferentes enfoques matemáticos y requisitos computacionales, el patrón Strategy encapsula estos algoritmos en clases separadas (CollaborativeFilteringStrategy, ContentBasedStrategy, HybridStrategy), permitiendo que el motor seleccione dinámicamente la estrategia más adecuada según el contexto del usuario, el tipo de producto o la campaña promocional activa. Considerando la necesidad de instanciar estratégicamente estos algoritmos según la configuración global o preferencias específicas del usuario, el patrón Factory Method en RecommendationStrategyFactory centraliza la creación de objetos RecommendationStrategy, encapsulando la lógica de selección y facilitando la incorporación de nuevos algoritmos de recomendación (como redes neuronales o modelos basados en grafos) sin modificar el código cliente que los consume. Para la persistencia y recuperación eficiente del catálogo de productos, el patrón Repository abstrae el acceso a la fuente de datos (actualmente una lista en memoria, pero preparado para migrar a bases de datos SQL, NoSQL o índices de búsqueda como Elasticsearch), proporcionando una interfaz consistente para operaciones de consulta que alimentan los algoritmos de recomendación. Ante la necesidad de integrar servicios externos de machine learning que pueden generar



predicciones más sofisticadas basadas en grandes volúmenes de datos, el patrón Adapter normaliza la comunicación envolviendo APIs externas no modificables (ExternalMLService) y proporcionando una interfaz uniforme (MLServiceAdapter) que el motor puede consumir independientemente del proveedor de ML, facilitando la incorporación de múltiples fuentes de inteligencia artificial. Para optimizar el rendimiento y evitar recomputaciones innecesarias, así como para mantener trazabilidad de las operaciones, el patrón Decorator permite añadir dinámicamente responsabilidades transversales al motor base (CacheDecorator para almacenar resultados previos, LoggingDecorator para registrar actividades) sin modificar la lógica central de recomendación, envolviendo objetos RecommendationEngine de manera transparente y permitiendo combinaciones flexibles según las necesidades del entorno de producción. En cuanto a la captura de eventos para análisis de negocio y retroalimentación del sistema, el patrón Observer desacopla la generación de recomendaciones de los mecanismos de notificación, permitiendo que cada recomendación generada notifique automáticamente a múltiples observadores (AnalyticsObserver para métricas de negocio, EmailObserver para campañas de marketing) sin acoplar el núcleo del motor a implementaciones específicas de análisis o comunicación. Finalmente, el patrón Facade en RecommendationFacade ofrece una interfaz simplificada que unifica todas estas capacidades (configuración global, selección de estrategia mediante fábrica, consulta de productos vía repositorio, aplicación de algoritmos estratégicos, integración con servicios ML mediante adaptador, decoración con cache y logging, y notificación a observadores), ocultando la complejidad subyacente y proporcionando un punto de entrada único que los equipos de desarrollo de frontend, aplicaciones móviles o servicios de personalización pueden consumir mediante una operación intuitiva como `get_recommendations(user_id)`. Esta arquitectura basada en patrones garantiza un motor de recomendación robusto, extensible y preparado para escalar conforme crezca el catálogo de productos y la base de usuarios, incorporando nuevos algoritmos de recomendación basados en deep learning, fuentes de datos en tiempo real, estrategias de cache distribuida y canales de análisis avanzados sin comprometer la estabilidad del núcleo del sistema.

Objetivo: Diseñar un sistema modular que combine múltiples algoritmos de recomendación.

Competencias específicas

- Diseñar algoritmos intercambiables.
- Encadenar filtros.
- Decorar recomendaciones.
- Centralizar configuración.

Contexto Empresa quiere:

- Recomendaciones por historial.



Universidad
Tecnológica
de Pereira

TALLER II
IS924 ACS
ING. OSCAR ANDRES SIERRA
JARAMILLO

- Recomendaciones por tendencia.
- Recomendaciones por perfil.

Requerimientos técnicos

- Permitir nuevos algoritmos.
- Encadenar filtros.
- Registrar comportamiento del usuario.

Patrones involucrados

- **Strategy** (algoritmos de recomendación)
- **Chain of Responsibility** (filtros)
- **Decorator** (personalización adicional)
- **Singleton** (gestor de configuración)

Preguntas de reflexión

- ¿Cómo permitir nuevos algoritmos?
- ¿Qué ventaja ofrece Decorator?
- ¿Por qué usar Chain of Responsibility?
- ¿Qué problema resuelve Singleton?
- ¿Cómo impacta en rendimiento y mantenimiento?